VisionMate: An AI-Powered Smart Spectacle System for Real-Time Object Recognition, Memory-Based Assistance, and Obstacle Avoidance for the Visually Impaired

Dhanush A S[1], Chirag M[2], Dhanush C K[3], Ganesh C[4] and Snehalatha N[5]

1,2,3,4,Department of Computer Science and Engineering, JSS Academy of Technical Education, Bengaluru – 560060, India

5 Assistant Professor, Department of Computer Science and Engineering, JSS Academy of Technical Education, Bengaluru – 560060, India (Project Guide)

Corresponding Authors List#:

Email: 1-dhanushas2056@gmail.com; USN: 1JS23CS046; Mobile: +91-7019249255

Email: 2-chirag07052005@gmail.com; USN: 1JS23CS038; Mobile: +91-9538314675

Email: 3-ckdhanush520@gmail.com; USN: 1JS23CS046; Mobile: +91-9113504932

Email: 4-ganeshac2006@gmail.com; USN: 1JS23CS046; Mobile: +91-6364105800

Funding or Supporting Agencies and Grants Details:

This work was completed as part of the Major Project curriculum of the Department of Computer Science and Engineering at JSS Academy of Technical Education, Bengaluru, under the guidance of Assistant Professor Snehalatha N. No external funding, grants, or sponsorship was received, and all hardware components were funded by the student authors.

ABSTRACT

People with visual impairments often depend on traditional aids such as white canes, which detect nearby obstacles but cannot identify objects, recognize familiar individuals, or remember personal belongings. This paper introduces VisionMate, a wearable smart-spectacle system that integrates real-time object detection, face recognition, personalized object memory, ultrasonic obstacle sensing, and priority-based audio feedback. By combining ultrasonic measurements with visual detections, the system provides distance-aware obstacle warnings, while a priority queue organizes events into spoken messages. A working prototype was assembled using an adapted 3D-printed spectacle frame, Raspberry Pi, camera module, and ultrasonic sensor, and evaluated through live integration testing. Results demonstrate object and person detection using YOLOv8n, with confidence scores above 0.8 for common objects, successful face recognition, and retrieval of enrolled objects through 1280-dimensional deep visual embeddings aggregated across multiple frames. The system achieved an end-to-end frame rate of 6.4–9.3 FPS on standard CPU hardware. GPS-based outdoor navigation and hands-free voice commands were implemented in software and tested in mock or offline mode, while integration with field hardware remains ongoing. These findings indicate that an affordable, integrated assistive device with personalized object memory can be developed using readily available components.

Keywords: Assistive Technology; Face Recognition; Object Detection; Object Memory; Obstacle Avoidance; Raspberry Pi; Text-to-Speech; Ultrasonic Sensor; Visually Impaired; Wearable Computing; YOLOv8.

1. INTRODUCTION

Visual impairment affects hundreds of millions of people, making it difficult to navigate unfamiliar places, locate objects, and recognize people. The traditional white cane remains a mobility aid, but it mainly detects obstacles through physical contact. It cannot explain what an obstacle is. For example, it cannot distinguish a chair from a person or help someone identify their water bottle among similar bottles on a table.

Advances in computer vision, deep learning, and compact computers have created opportunities for wearable assistive devices that do more than detect obstacles. However, the related work reviewed in Section 2 shows that many solutions focus on one task, such as obstacle avoidance, object detection, or navigation. These functions are rarely combined. Personalized memory is also uncommon: many devices recognize general object categories but cannot learn an object or person associated with a user and identify it again later.

This paper presents VisionMate, a wearable smart-spectacle system that brings four capabilities together: (i) real-time detection of everyday objects, (ii) face recognition for familiar individuals, (iii) a remember-and-find feature that stores and retrieves personal objects using deep visual embeddings, and (iv) obstacle warnings combining ultrasonic sensing with priority-based audio feedback. These functions provide information about the surroundings and help users respond to hazards.

VisionMate uses components, including a Raspberry Pi, a camera module, an ESP32 microcontroller, and an ultrasonic sensor. A companion computer performs intensive deep-learning inference and communicates with the wearable unit wirelessly. This arrangement reduces the processing burden on the device and supports a lightweight, low-cost design. The prototype combines visual recognition, personal object retrieval, and distance-based obstacle alerts without requiring intensive processing directly on the spectacles.

The remainder of this paper is organized as follows. Section 2 reviews related studies. Section 3 explains the system architecture and implementation of each module. Section 4 reports experimental results from the working prototype. Section 5 discusses the findings. Section 6 outlines future improvements, and Section 7 concludes the paper.

2. LITERATURE REVIEW

Several assistive navigation systems have been introduced in recent years to support people with visual impairments. Li et al. developed a vision-based indoor navigation aid that uses a smartphone camera to help blind users determine their location and avoid obstacles [1]. Wearable systems equipped with ultrasonic sensors have also been explored to detect nearby obstacles under different lighting conditions. However, these systems generally indicate the presence of an obstacle and its distance without identifying the object itself [2]. Barontini et al. investigated wearable haptic feedback combined with obstacle avoidance for indoor navigation, demonstrating the benefits of combining different feedback methods beyond audio alone [3].

Object detection models based on deep learning, including YOLO, have also been incorporated into assistive smart glasses to identify objects within the user's field of view. This approach adds semantic information to detected obstacles, addressing a limitation of ultrasonic-only systems. Some implementations combine a Raspberry Pi or ESP32-class controller, a camera, and a YOLO model to support real-time detection and audio feedback [4], [5]. Other systems extend detection beyond the immediate field of view by using a second wearable device, such as instrumented shoes, to improve obstacle coverage [6]. Lightweight ESP32-CAM-based YOLO pipelines have also been explored to reduce hardware costs and power consumption [7]. Research on currency recognition for visually impaired users demonstrates how fine-tuning a general object detector can improve performance for specific applications [8]. Islam et al. developed smart glasses that combine scene understanding with speech-based interaction, allowing users to ask questions about their surroundings [10].

The systems discussed above rely on established deep-learning architectures. YOLO models perform single-shot object detection, while lightweight convolutional neural networks, such as MobileNetV2, provide practical options for running inference on resource-constrained devices [11], [12].

Despite these developments, personalized object recognition remains an important gap. Existing systems can announce that a bottle is nearby, but they generally cannot identify which bottle belongs to the user or remember a previously registered object for later retrieval.

3. METHODOLOGY

VisionMate follows a split-compute architecture in which lightweight sensing and actuation occur on the wearable unit while computationally expensive deep-learning inference is offloaded to a paired companion machine over a local wireless link. This design keeps the head-worn hardware small, light, and low-power, while still allowing the use of full-sized object-detection and face-recognition models that would be too slow to run in real time on the Raspberry Pi alone.

3.1. System Architecture

As shown in Fig. 1, the architecture is organized into three logical layers. The Input/Wearable Layer consists of a Raspberry Pi 4B fitted with a Pi camera module that captures the video stream, while an ESP32 microcontroller reads an HC-SR04 ultrasonic sensor and a push button and reports readings to the Raspberry Pi over a serial USB link; audio output is delivered through Bluetooth earbuds. The Processing Layer transmits video frames, ultrasonic distances, and button events over Wi-Fi (WebSocket, port 8080) to a companion laptop, which runs the object detector, face recognizer, object-memory matcher, spatial navigator, obstacle-fusion module, GPS/OSM navigation engine, and the voice-command and priority-based text-to-speech engines. The Output/API Layer is a Flask-based REST API (ports 5000/5001) that exposes endpoints such as /process_frame, /enroll_face, and /query_item, and serves a lightweight mobile/caretaker backend that reports the wearer's location, forwards notifications, and streams webcam snapshots. All persistent data — enrolled faces, enrolled objects and their deep embeddings, and object sighting logs — are stored in a local SQLite database, keeping the system fully functional without a permanent internet connection except where GPS routing explicitly requires map data.

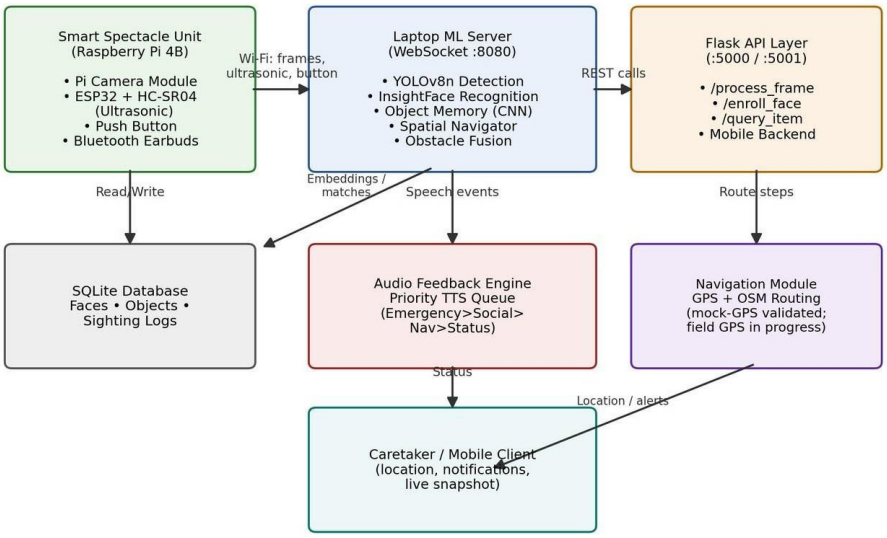


Fig. 1. VisionMate system architecture: the wearable input layer streams data over Wi-Fi to a laptop-based ML processing layer, which returns detections, matches, and navigation guidance converted into speech, while a Flask API layer exposes state to an optional caretaker/mobile client.

3.2. Real-Time Object Detection

Object detection is performed using a YOLOv8-nano model (yolov8n.pt) chosen for its favorable accuracy-to-speed trade-off on CPU hardware. Detection is restricted to a configurable set of target classes relevant to daily assistive use — person, chair, bottle, couch, tv, keys, cell phone, and clock — at a confidence threshold of 0.5 and an IoU threshold of 0.45. To sustain real-time throughput on CPU, inference is run on a reduced 320 x 320 input resolution, optionally accelerated through Intel OpenVINO on

the integrated GPU/NPU when available, and is executed only on every third captured frame (frame skipping), with intermediate frames re-using the most recent detection result.

3.3. Face Recognition

Face detection and recognition use the InsightFace buffalo_l model [13], which produces a face embedding compared against enrolled faces in the database using a Euclidean-distance threshold (0.65). When a face is recognized within a cooldown window, the system announces the person's name through the audio-feedback queue (e.g., "Hello, Dhanush AS"). If InsightFace is unavailable, the system automatically falls back to an OpenCV Haar-cascade detector for detection-only operation.

3.4. Memory-Based Object Recognition (Remember/Find)

One of VisionMate's main features is its object-memory module, which enables users to register a particular personal belonging instead of relying only on its general object category. When the user activates enrollment using a button or the "remember this" voice command while a YOLO-detected object is visible, the system carries out the following steps

1) Crops the detected bounding-box region across ten consecutive frames;

2) Extracts a 1280-dimensional deep visual embedding from each crop using a MobileNetV2 backbone (ImageNet-pretrained, classifier head removed) rather than only classical ORB keypoint features;

3) Aggregates the ten frame-level embeddings into a single averaged embedding, improving robustness to lighting and viewpoint variation; and

4) Stores the embedding, the YOLO class, and a user-supplied label (e.g., "MyBottle") in the SQLite database.

During normal operation, whenever an object of a previously enrolled class is detected, its embedding is compared against stored candidates using cosine similarity, and a match is reported when similarity exceeds an empirically tuned threshold, shown in Equation-1, chosen as a compromise point between observed genuine-match similarities (0.40-0.71) and the need to avoid false positives.

$$(A, B) = (A \cdot B) / (\|A\| \|B\|) \geq 0.62 \qquad \textbf{Equation-1}$$

Similarity matches are announced through audio feedback (e.g., "MyBottle, 68 percent confidence") and logged with a timestamp so that a later "find my bottle" query can report where and when the object was last seen.

3.5. Obstacle Fusion and Spatial Navigation

Bounding-box-based distance estimation from a single camera is unreliable at close range, so VisionMate fuses ultrasonic ranging with visual detections: for any detected object that falls within the ultrasonic sensor's 15-degree field of view at the centre of the 60-degree camera field of view, the corresponding vision-based bounding box is annotated with the sensor's precise range reading rather than a bounding-box-derived estimate. Independently, a spatial navigator divides each frame into a 3x3 zone grid and assigns priority to obstacles by zone (bottom zones — the immediate path — given highest priority), issuing directional guidance such as "person ahead, move left" for obstacle classes including people, furniture, and vehicles.

3.6. Priority-Based Audio Feedback

All system outputs — obstacle warnings, face/object recognition announcements, navigation instructions, and status messages — are routed through a single priority queue with four levels (Emergency > Social > Navigational > Status), ensuring that a collision warning always pre-empts a lower-priority announcement such as a recognized face. Speech synthesis uses the offline pyttsx3 engine by default, with an online gTTS fallback available.

3.7. Navigation and Voice Control

The codebase additionally implements GPS-based turn-by-turn navigation using OpenStreetMap routing (with an optional Google Maps back-end) and a speech-recognition-based voice-command interface for hands-free operation (e.g., "remember this," "find my keys," "where am I"). Both modules have been implemented and validated using a mock-GPS mode and offline test scripts; integration with a physical outdoor GPS receiver and an on-frame microphone is in progress and is discussed further in Section 6.

3.8. Hardware and Software Implementation

Table 1 lists the hardware used to build the working prototype, shown in Fig. 2. The camera and ultrasonic sensor are mounted on a repurposed spectacle frame, with the Raspberry Pi and a breadboard carrying the ESP32 and sensor wiring worn or carried by the user.

Table -1: Hardware Components of the VisionMate Prototype

Component	Role
Raspberry Pi 4B	On-frame video capture and Wi-Fi link
Pi Camera Module	Real-time video input (mounted on frame)
ESP32 Dev Board	Reads ultrasonic sensor and button, sends to Pi
HC-SR04 Ultrasonic Sensor	Distance measurement for obstacle fusion
Push Button	Manual trigger for object/face enrollment
Bluetooth Earbuds	Audio feedback delivery
Power Bank (10,000 mAh)	Portable power for the wearable unit
Laptop (companion device)	Runs YOLOv8, InsightFace, and TTS engines

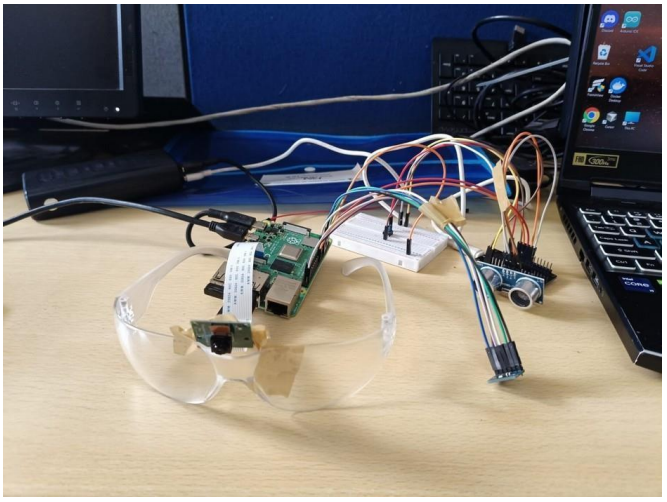


Fig. 2. Assembled VisionMate hardware prototype: Raspberry Pi 4B, camera module, and ultrasonic sensor wired via a breadboard and mounted on a spectacle frame.

The processing layer is implemented in Python 3.10+. Object detection uses Ultralytics YOLOv8, face recognition uses InsightFace with ONNX Runtime, and object-memory embeddings use PyTorch/torchvision MobileNetV2. Image handling uses OpenCV. The database layer uses SQLite (via Python's built-in sqlite3 module). The API layer uses Flask with Flask-CORS, and text-to-speech uses pyttsx3/gTTS. Table 2 summarizes the key libraries used.

Table -2: Key Software Libraries and Their Role

Library	Used For
ultralytics (YOLOv8)	Real-time generic object detection
insightface + onnxruntime	Face detection and recognition
torch / torchvision	MobileNetV2 deep embeddings (object memory)

Library	Used For
opencv-python	Frame capture, cropping, image operations
scikit-learn	Similarity / matching utilities
Flask, Flask-CORS	REST API for detections and mobile backend
pyttsx3, gTTS	Offline / online text-to-speech
SQLite3	Persistent storage of faces, objects, sightings

The Raspberry Pi streams JPEG-encoded frames and sensor readings to the laptop over a WebSocket connection at approximately 10 Hz for sensor data, while the vision pipeline processes frames as fast as the detection cadence allows. This wireless split-compute design was chosen deliberately so that the wearable hardware remains light and battery-efficient while the full-sized deep models run on the more capable companion device.

4. RESULT

The prototype was evaluated through live integration testing using the pipeline described above, with the camera and ultrasonic sensor active and the laptop server processing frames in real time.

4.1. Object Detection and Face Recognition

Fig. 3 shows a representative frame in which the system simultaneously detects a person (confidence 0.94) and recognizes the enrolled face as "Dhanush AS" with a similarity score of 0.35-0.65 across different frames/lighting conditions, while the on-screen navigation cue instructs the user to "move straight." Generic object classes such as bottle were reliably detected at confidences above 0.8 under normal indoor lighting.

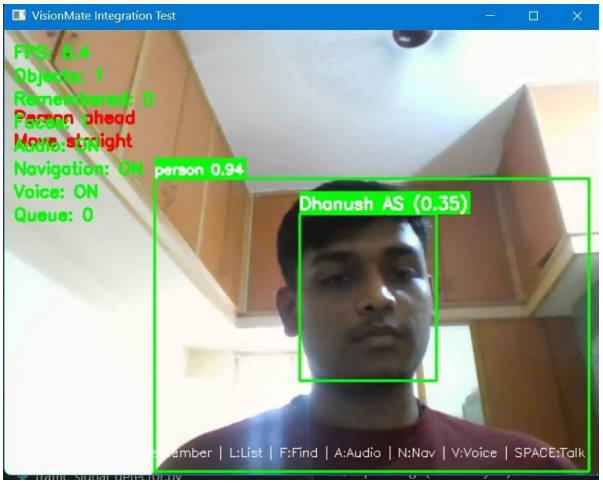


Fig. 3. Live integration test: simultaneous person detection (YOLOv8, confidence 0.94) and face recognition ("Dhanush AS"), with on-screen FPS, navigation, and audio status indicators.

4.2. Object Memory: Enrollment and Recognition (Before vs. After)

The remember/find workflow was validated end-to-end using a before/after comparison across the enrollment (database-write) step, to directly show what changes once an object is saved to the object-memory database. Before enrollment, the target bottle carried no personal identity: it was detected only as a generic YOLO object, labeled "bottle" at confidence 0.81, with the on-screen Remembered counter reading 0, as shown in Fig. 4. The user then triggered enrollment through the on-screen "Remember" command; the object was cropped across five consecutive frames and aggregated into a single 1280-dimensional MobileNetV2 embedding, as confirmed by the system log ("Aggregated deep embeddings from 5 frames: 1280D vector — Object 'MyBottle' enrolled successfully"), and persisted to the SQLite database under the custom label "MyBottle."

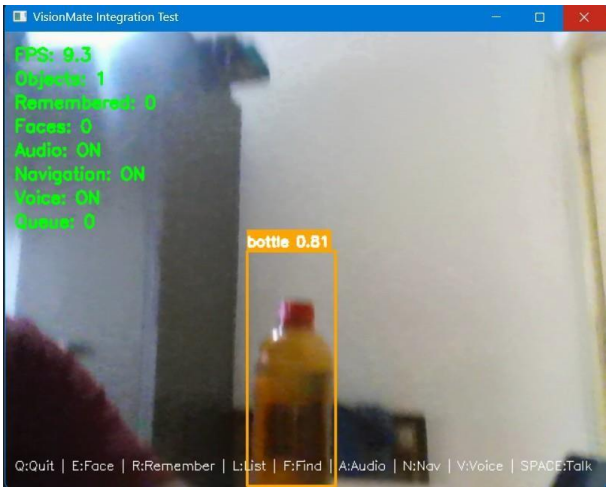


Fig. 4. Before enrollment: the bottle is detected only as a generic YOLO "bottle" object (confidence 0.81), with Remembered: 0, indicating no personalized match yet exists in the database.

After the embedding was written to the database, the same physical bottle was correctly re-identified in subsequent frames with reported match confidences in the 62-68% range, consistent with the configured 0.62 similarity threshold. The system now distinguished the personally labeled bottle from the generic YOLO "bottle" class by tagging it with a magenta "MyBottle" label and match percentage alongside the standard green YOLO detection box, as illustrated in Fig. 5, which shows a 70% match confidence immediately after enrollment. This before/after pairing confirms that the personalized memory layer activates only once an object has been saved to the database, and correctly falls back to generic category-level detection for any object not yet enrolled.

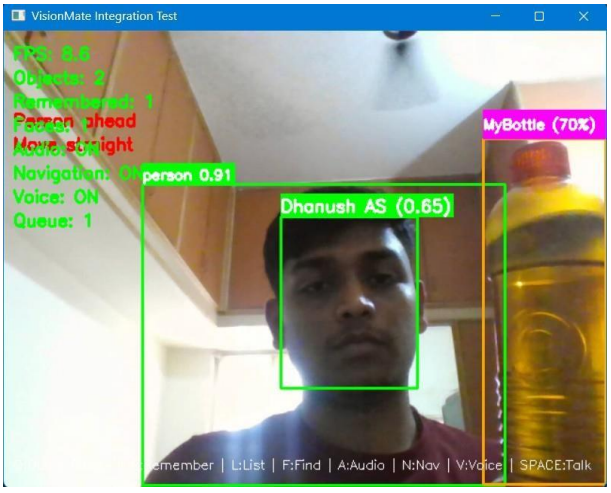


Fig. 5. After enrollment: simultaneous recognition of a known face ("Dhanush AS") and the previously enrolled personal object ("MyBottle," 70% match confidence), demonstrating the memory-based recognition pipeline.

4.3. Runtime Performance

Table 3 reports per-stage processing latency captured from the live integration logs. End-to-end throughput of 6.4-9.3 FPS was observed on CPU-only inference, which is adequate for a walking-pace assistive use case, though YOLO inference latency is the dominant and most variable cost, driven by system load and the OpenVINO acceleration path.

Table -3: Observed Per-Stage Processing Latency (Milliseconds)

Pipeline Stage	Latency Range (ms)
Frame capture (cap.read)	9.4 - 15.0
YOLOv8 object detection	10.2 - 85.5
Face recognition (InsightFace)	0.3 - 58.4
Object-memory matching	0.0 - 0.1
Navigation processing	0.0 - 0.3

Pipeline Stage	Latency Range (ms)
Display / overlay rendering	2.6 - 8.5
Observed end-to-end frame rate	6.4 - 9.3 FPS

4.4. Obstacle Warning

Fusing ultrasonic ranging with visual detections resolved the ambiguity of monocular bounding-box distance estimates: when an object fell inside the sensor's forward cone, the reported distance corresponded to a direct physical measurement rather than an estimate based on apparent bounding-box size, giving consistent close-range obstacle warnings during indoor testing.

5. DISCUSSION

The results confirm that a personalized, memory-based recognition layer can be added on top of a generic detector without materially harming real-time performance: object-memory matching added a negligible 0.0-0.1 ms to the pipeline (Table 3) because it operates on a single pre-computed embedding rather than re-running the CNN backbone. This addresses the gap identified in Section 2, where existing assistive systems can announce a generic object category but cannot distinguish the user's own belongings from visually similar items.

The reported similarity range for genuine matches (0.62-0.68 for the enrolled bottle, against a 0.62 threshold) indicates that the current threshold is set close to the lower bound of observed genuine similarities, which favors recall over precision. In practical use this trades a small risk of false-positive matches for a lower risk of the system failing to recognize a personally enrolled object, which the authors consider the more acceptable failure mode for an assistive device. Face-recognition similarity scores (0.35-0.65) were more variable across lighting conditions than object-memory scores, suggesting that face recognition is more sensitive to illumination than the MobileNetV2-based object embeddings, consistent with prior literature on face recognition robustness.

The end-to-end frame rate of 6.4-9.3 FPS on CPU-only hardware is adequate for a walking-pace assistive scenario but would benefit from further acceleration; YOLO inference remained the dominant and most variable cost, which motivates the hardware-acceleration direction discussed in Section 6.

6. FUTURE SCOPE

While the core recognition and obstacle-fusion pipeline is fully functional, three aspects of the system remain work in progress at the time of writing. GPS Navigation: turn-by-turn OSM-based routing logic is implemented and has been validated using a mock-GPS/manual-override mode; integration with a physical outdoor GPS receiver on the wearable unit and field testing of live routing have not yet been completed. Voice Command Interface: speech-recognition-driven hands-free control (e.g., spoken "remember this" / "find my keys") is implemented in software; deployment with a frame-mounted microphone for reliable far-field speech capture is still being integrated. Environmental Sensitivity: as with any camera-based system, detection and recognition accuracy degrade under poor or excessive lighting, and CPU-only inference limits the maximum sustainable frame rate compared to GPU-accelerated deployments.

Beyond these immediate next steps, future work will focus on: (i) extending face recognition to support larger enrolled-face databases with lower latency; (ii) adding an emergency-alert feature that notifies a caregiver's phone with the user's location in unsafe situations; (iii) developing a companion mobile application for object/face management and remote monitoring; and (iv) porting the vision pipeline to run directly on the Raspberry Pi using a hardware accelerator (e.g., a Coral TPU or the Pi's NPU-capable successor) to remove the dependency on a paired laptop.

7. CONCLUSION

This paper presented VisionMate, a wearable smart-spectacle assistive system that integrates real-time object detection, face recognition, a novel personal object-memory module based on aggregated deep embeddings, ultrasonic-fused obstacle warning, and priority-based audio feedback on a low-cost hardware platform. Live integration testing of the working prototype confirmed reliable detection and recognition performance — including successful enrollment and re-identification of a user-specific object with 1280-dimensional MobileNetV2 embeddings at 62-68% match confidence — at an interactive frame rate of 6.4-9.3 FPS on commodity CPU hardware. By combining generic object detection with personalized, memory-based recognition, VisionMate addresses a gap identified in existing assistive-navigation literature, where systems typically support only generic obstacle or object detection without the ability to learn and recall a user's own belongings or acquaintances.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to their project guide, Snehalatha N, Assistant Professor, Department of Computer Science and Engineering at JSS Academy of Technical Education, Bengaluru, for her valuable guidance, constructive feedback, and support throughout the design and development of the project. The authors also thank the Department of Computer Science and Engineering for providing the necessary resources and facilities to develop and test the prototype successfully.

REFERENCES

[1] B. Li, J. P. Munoz, X. Rong, Q. Chen, J. Xiao, Y. Tian, and M. Yousuf, "Vision-based mobile indoor assistive navigation aid for blind people," IEEE Trans. Mobile Computing, vol. 18, no. 3, pp. 702-714, 2019.

[2] X. Ma, X. Chen, and W. Xiong, "Efficient obstacle detection for wearable device-based guidance system," in Proc. 2019 IEEE Int. Conf. Consumer Electronics (ICCE), 2019, pp. 1-5.

[3] F. Barontini, M. G. Catalano, L. Pallottino, B. Leporini, and M. Bianchi, "Integrating wearable haptics and obstacle avoidance for the visually impaired in indoor navigation: A user-centered approach," IEEE Trans. Haptics, vol. 14, no. 1, pp. 109-122, 2021.

[4] "Smart Glasses for Disabled People," in Proc. 2023 IEEE Conf., IEEE Xplore, doc. 10448713.

[5] R. P. Narwaria, A. Ahirwar, A. K. Prajapati, A. Kumar, and A. K. Tiwari, "Smart Object Detection Using ESP32-CAM Based on YOLO Algorithm," in Proc. 2024 Int. Conf. Innovative Computing and Communication (ICoICI), 2024, doi: 10.1109/ICoICI62503.2024.10696374.

[6] "AI-Powered Smart Glasses and Shoes for the Visually Impaired," in IEEE Conference Publication, IEEE Xplore, doc. 10883641.

[7] S. Channagoudar, M. B. Prakash, H. M. Harish, and S. Udara, "Deep learning-based object detection for assisting visually impaired people," NeuroQuantology, vol. 20, no. 5, 2022, doi: 10.48047/nq.2022.20.5.nq22835.

[8] R. C. Joshi, S. Yadav, and M. K. Dutta, "YOLOv3 based currency detection and recognition system for visually impaired persons," in Proc. 2020 Int. Conf. Contemporary Computing and Applications (IC3A), IEEE, 2020, pp. 280-285.

[9] M. T. Islam, M. A. Rashid, M. Ahmad, A. Kuwana, and H. Kobayashi, "Design and implementation of smart guided glass for visually impaired people," Int. J. Electrical and Computer Engineering (IJECE), vol. 9, no. 4, pp. 3345-3355, 2020.

[10] C. -H. Chen and M.-F. Shiu, "Smart guiding glasses with descriptive video service and spoken dialogue system for visually impaired," in Proc. 2020 IEEE Int. Conf. Consumer Electronics - Taiwan (ICCE-Taiwan), 2020, pp. 1-2.

[11] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767, 2018.

[12] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE/CVF CVPR, 2018, pp. 4510-4520.

[13] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in Proc. IEEE/CVF CVPR, 2019, pp. 4690-4699.

AUTHORS DETAILS

The authors' information is provided below in the order it appears on the manuscript, followed by the designated corresponding author. Each author's photograph is included as required by the NCAIT-2026 submission format.

First Author

Name	Dhanush A S
Email	dhanushas2056@gmail.com
Phone	+91 7019249255
USN	1JS23CS046
Current Affiliation	Undergraduate (UG) Student, B.E. Computer Science and Engineering
Current Institute	JSS Academy of Technical Education, Bengaluru, India
Institute Address	JSS Academy of Technical Education, Dr. Vishnuvardhan Road, Uttarahalli–Kengeri Main Road, Srinivaspura, Bengaluru, Karnataka 560060.



Second Author

Name	Chirag M
Email	chirag07052005@gmail.com
Phone	+91 9538314675
USN	1JS23CS038
Current Affiliation	Undergraduate (UG) Student, B.E. Computer Science and Engineering
Current Institute	JSS Academy of Technical Education, Bengaluru, India
Institute Address	JSS Academy of Technical Education, Dr. Vishnuvardhan Road, Uttarahalli–Kengeri Main Road, Srinivaspura, Bengaluru, Karnataka 560060.



Third Author

Name	Dhanush C K
Email	ckdhanush520@gmail.com
Phone	+91 9113504932
USN	1JS23CS047
Current Affiliation	Undergraduate (UG) Student, B.E. Computer Science and Engineering
Current Institute	JSS Academy of Technical Education, Bengaluru, India
Institute Address	JSS Academy of Technical Education, Dr. Vishnuvardhan Road, Uttarahalli–Kengeri Main Road, Srinivaspura, Bengaluru, Karnataka 560060.



Fourth Author

Name	Ganesh C	
Email	ganeshac2006@gmail.com	
Phone	+91 6364105800	
USN	1JS23CS054	
Current Affiliation	Undergraduate (UG) Student, B.E. Computer Science and Engineering	
Current Institute	JSS Academy of Technical Education, Bengaluru, India	
Institute Address	JSS Academy of Technical Education, Dr. Vishnuvardhan Road, Uttarahalli–Kengeri Main Road, Srinivaspura, Bengaluru, Karnataka 560060.	

Project Guide

Name	Snehalatha N
Designation	Assistant Professor, Dept. of Computer Science and Engineering
Current Institute	JSS Academy of Technical Education, Bengaluru, India
Role	Project Guide

Corresponding Author

Dhanush A S (dhanushas2056@gmail.com, Phone: +91 7019249255, USN: 1JS23CS046)